

UNIT INFORMATION	
COURSE – (LEADING TO)	BSC (HONS) IN SOFTWARE ENGINEERING
UNIT NAME	Business Component Development I
UNIT ID	JIAT/BCD I
ASSIGNMENT ID	JIAT/BCD I/EX/01

STUDENT INFORMATION	
STUDENT NAME	Menara Sathmini Perera
STUDENT ID	200582800963
BATCH	COHORT 8
BRANCH	Colombo

Table of Contents

2.1 System Architecture Overview....3
2.2 Implementation Specifications and Optimization Techniques 5
2.3 Configuration Documentation with Performance Tuning Parameters 9
2.4 Deployment architecture with performance considerations....10
2.5 Session bean implementations with lifecycle management strategies 10
2.5 Messaging System Design with Throughput Optimization 15
2.6 Database integration with connection optimization....17
2.7 Asynchronous processing with performance measurement results....17
2.8 Critical analysis of Java EE platform selection with comparative evaluation 17
2.9 Session bean design decisions with lifecycle optimization analysis....17
2.10 JNDI and dependency injection strategy with performance impact assessment 18
2.11 Asynchronous processing implementation with reliability analysis....19
2.12 JMS messaging architecture with scalability evaluation 20
2.13 Message-driven bean design with lifecycle optimization analysis....20

2. Technical Implementation Documentation

2.1 System Architecture Overview

Tehcmart plastofroms use a modular multi-tier Jakarta EE architecture designed for easy scalling and high performance, we have divided the project into five specific Maven Modules, each handling a different part of system.

- **Techmart-core:** This is where the main data (Entities) and DTOs needed by the system are located. This helps all other modules.
- **Techmart-ejb:** This is the 'brain' of the system. All business logic, calculations, and database operations happen here.
- **Techmart-web**: This is what controls the system's interface
- **Techmart- payment:** This is a special section dedicated solely to payment processing. Having this separate section is very good for security.
- **Techmart-ear:** This EAR is a compilation of all four modules and is designed to be uploaded to the server as a single file.

Component Relationship and Scalability Analysis:

The system employs a modular architecture using dependency injection to decouple the web and EJB tiers. Database interactions are optimized via JPA and **Tech mart** which eliminates connection overhead. By decoupling business and payment modules, the system supports independent scalability. Furthermore, **Payara 6** integrates asynchronous processing for non-blocking tasks, ensuring high responsiveness during peak traffic.

WebSocket Integration and Real-time Communication:

To enable instantaneous updates, we implemented **Jakarta WebSocket endpoints** within the techmart-ejb module. This eliminates polling latency by allowing server-pushed communication. By synchronizing this layer with our **JMS messaging infrastructure**, we provide clients with consistent, real-time data, significantly enhancing the user experience.

2.2Implementation Specifications and Optimization Techniques

The business logic implantation using session bean types,

- @Stateless Used for frequent task that don't require data retention such as logging in and searching for products.
- @Stateful it was used for tasks like shopping cart, where a user's information needs to be remembered until the end of the session.
- @Singleton is used for tasks that need to be inited only once at system startup

JMS Messaging: We use JMS to exchange messages. We use a Queue to confirm an order and a Topic to send stock alerts.

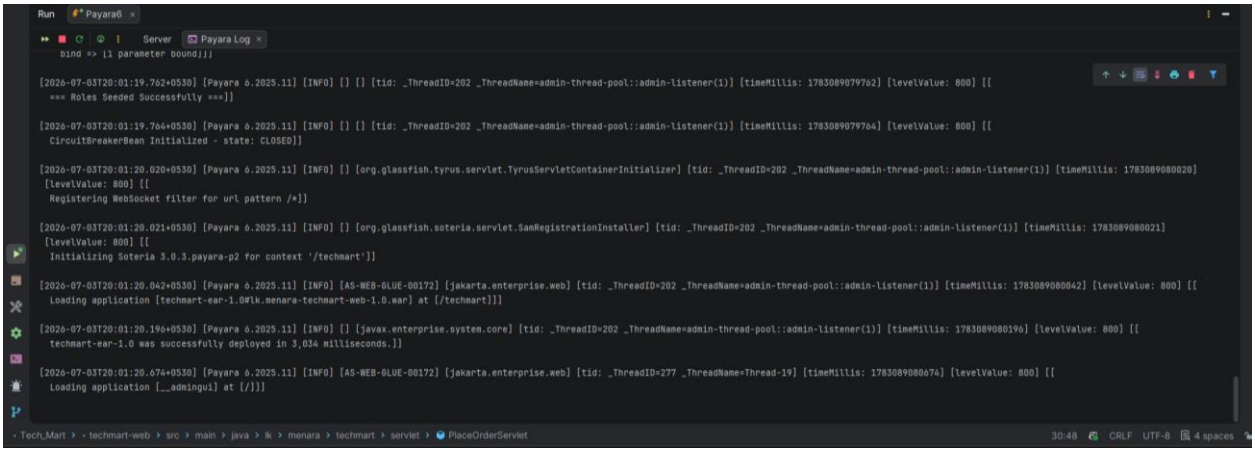
Database: To access the database, we will connect to MySQL using JPA and TechMartPool.

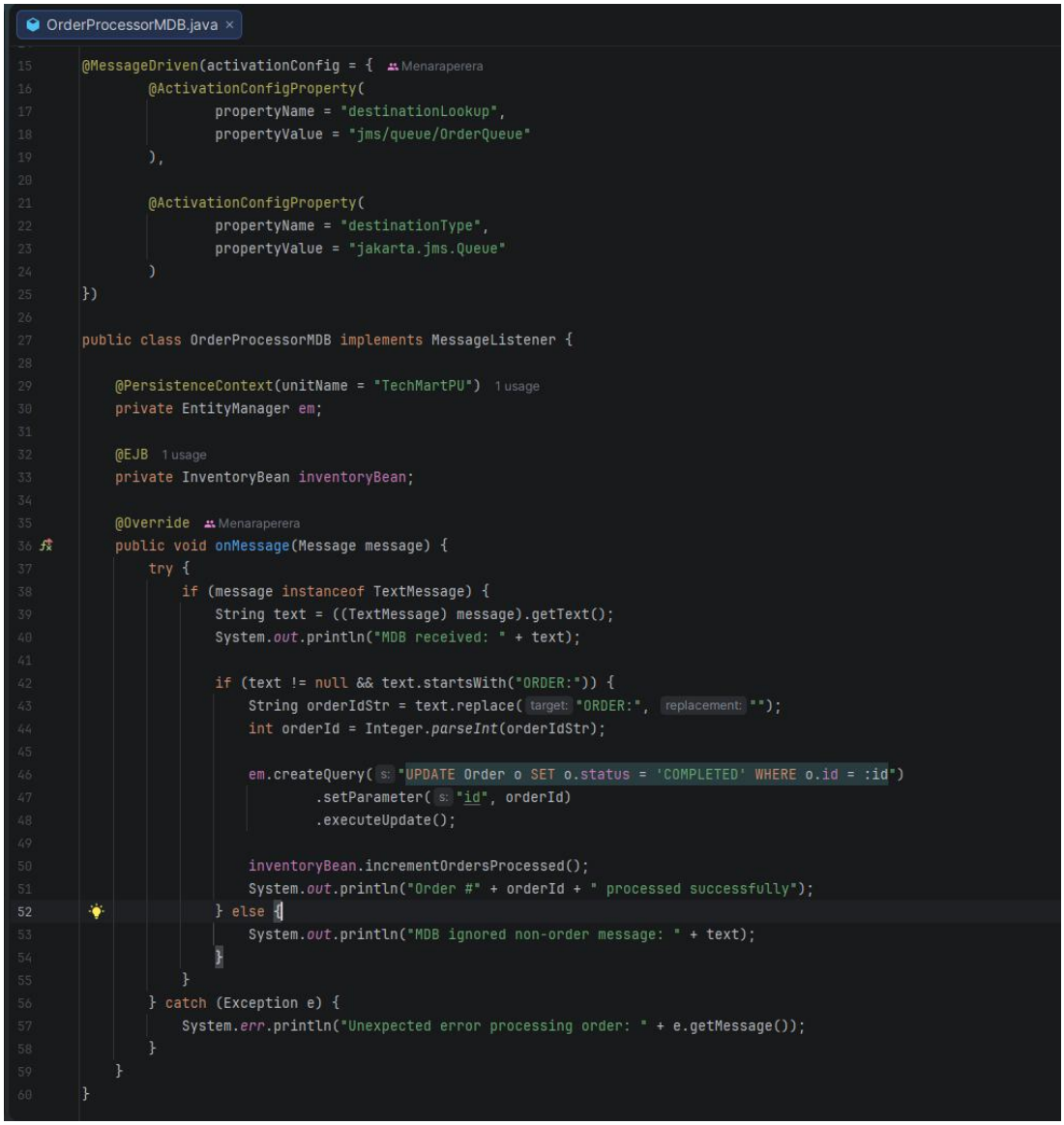
Optimization Techniques

- We use pre created database connections from TechMartPool instead of creating new ones. This greatly reduced the load on the server.
- Tasks that can block the system, such as sending emails, are done in the background using @Asynchronous. This makes the system feel very fast to the user.
- MDBs are used to process orders during peak hours. These work in the background without interfering with other tasks.
- Due to the bean pool on the Payara server, the same bean can be used by many people. This saves memory.

Fault Tolerance with Circuit Breaker Pattern

To keep the system stable, we added a **Circuit Breaker**. If the Payment Gateway fails, this stops the system from crashing and protects the application by blocking requests to the faulty service immediately





```java
@MessageDriven(activationConfig = {  Menaraperera
        @ActivationConfigProperty(
                propertyName = "destinationLookup",
                propertyValue = "jms/queue/OrderQueue"
        ),

        @ActivationConfigProperty(
                propertyName = "destinationType",
                propertyValue = "jakarta.jms.Queue"
        )
})

public class OrderProcessorMDB implements MessageListener {

    @PersistenceContext(unitName = "TechMartPU")  1 usage
    private EntityManager em;

    @EJB  1 usage
    private InventoryBean inventoryBean;

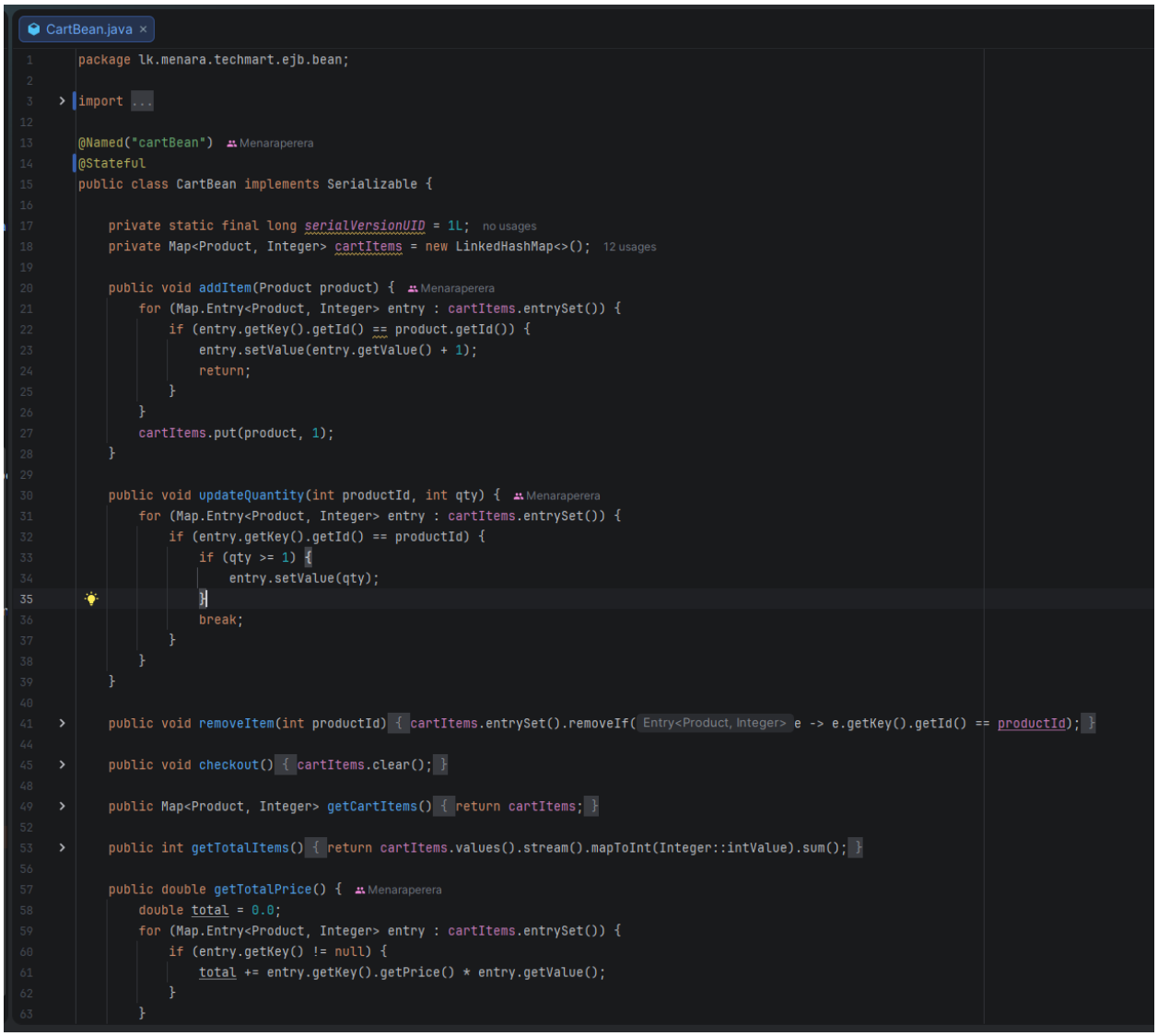
    @Override  Menaraperera
    public void onMessage(Message message) {
        try {
            if (message instanceof TextMessage) {
                String text = ((TextMessage) message).getText();
                System.out.println("MDB received: " + text);

                if (text != null && text.startsWith("ORDER:")) {
                    String orderIdStr = text.replace( target: "ORDER:",  replacement: "");
                    int orderId = Integer.parseInt(orderIdStr);

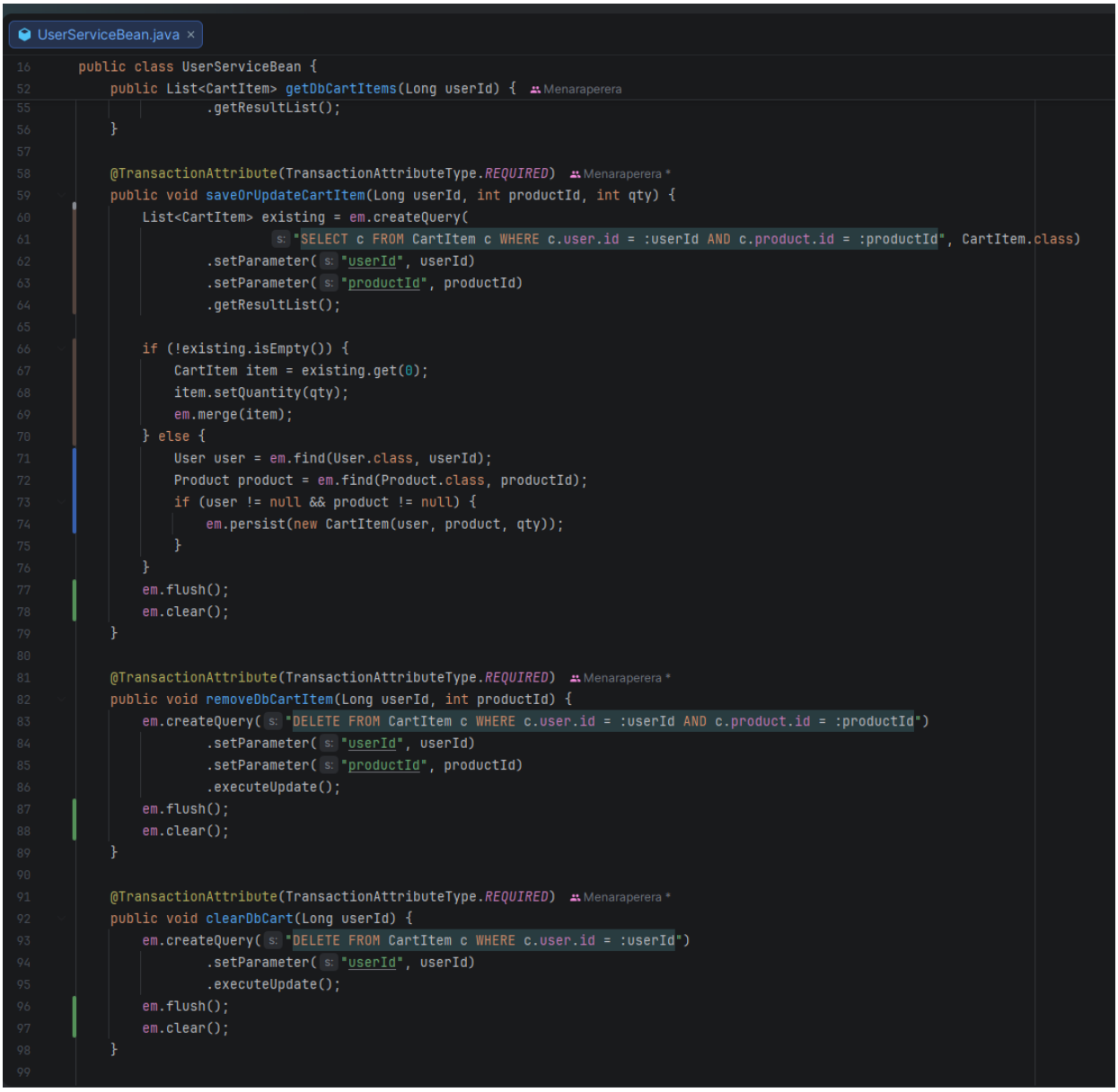
                    em.createQuery( s: "UPDATE Order o SET o.status = 'COMPLETED' WHERE o.id = :id")
                            .setParameter( s: "id", orderId)
                            .executeUpdate();

                    inventoryBean.incrementOrdersProcessed();
                    System.out.println("Order #" + orderId + " processed successfully");
                } else {
                    System.out.println("MDB ignored non-order message: " + text);
                }
            }
        } catch (Exception e) {
            System.err.println("Unexpected error processing order: " + e.getMessage());
        }
    }
}
```

The web tier keeps the system running smoothly by using a **JMS OrderQueue** instead of updating the database directly. When a user places an order, it is sent to this queue, which acts like a buffer. This prevents the system from crashing during busy times because the system can process orders one by one at a steady speed. Also, if there is a database error, the JMS system automatically tries again to save the order, ensuring that no data is lost and the system remains stable.



We optimized shopping cart operations by implementing the CartBean with a LinkedHashMap, which enables in-memory item storage and efficient lookups. By managing product quantities directly in memory and bypassing redundant database calls, we significantly reduce latency during frequent cart modifications. This design ensures a responsive, seamless user experience while minimizing resource overhead under concurrent usage



```java
public class UserServiceBean {
    public List<CartItem> getDbCartItems(Long userId) {  Menaraperera
            .getResultList();
    }

    @TransactionAttribute(TransactionAttributeType.REQUIRED)  Menaraperera *
    public void saveOrUpdateCartItem(Long userId, int productId, int qty) {
        List<CartItem> existing = em.createQuery(
                s: "SELECT c FROM CartItem c WHERE c.user.id = :userId AND c.product.id = :productId", CartItem.class)
            .setParameter( s: "userId", userId)
            .setParameter( s: "productId", productId)
            .getResultList();

        if (!existing.isEmpty()) {
            CartItem item = existing.get(0);
            item.setQuantity(qty);
            em.merge(item);
        } else {
            User user = em.find(User.class, userId);
            Product product = em.find(Product.class, productId);
            if (user != null && product != null) {
                em.persist(new CartItem(user, product, qty));
            }
        }
        em.flush();
        em.clear();
    }

    @TransactionAttribute(TransactionAttributeType.REQUIRED)  Menaraperera *
    public void removeDbCartItem(Long userId, int productId) {
        em.createQuery( s: "DELETE FROM CartItem c WHERE c.user.id = :userId AND c.product.id = :productId")
            .setParameter( s: "userId", userId)
            .setParameter( s: "productId", productId)
            .executeUpdate();
        em.flush();
        em.clear();
    }

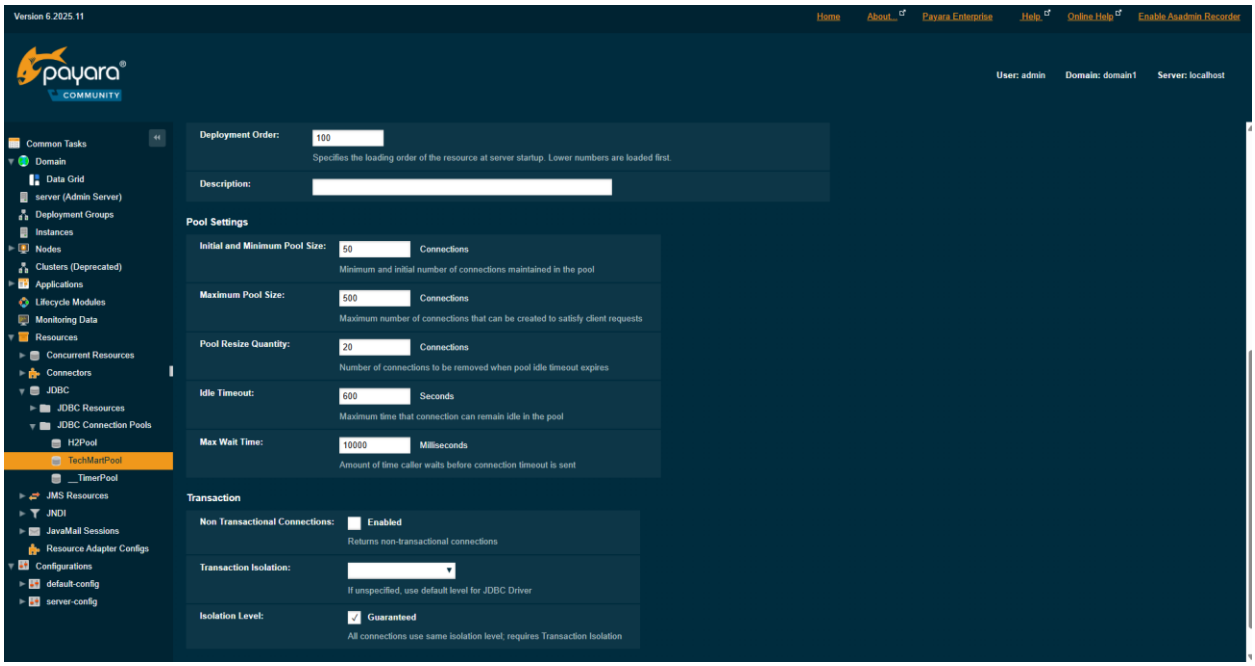
    @TransactionAttribute(TransactionAttributeType.REQUIRED)  Menaraperera *
    public void clearDbCart(Long userId) {
        em.createQuery( s: "DELETE FROM CartItem c WHERE c.user.id = :userId")
            .setParameter( s: "userId", userId)
            .executeUpdate();
        em.flush();
        em.clear();
    }
```

User authentication and profile management are handled by the Stateless UserServiceBean. This approach allows the EJB container to pool instances, enabling the system to scale effectively and handle a high volume of concurrent authentication requests without performance degradation.

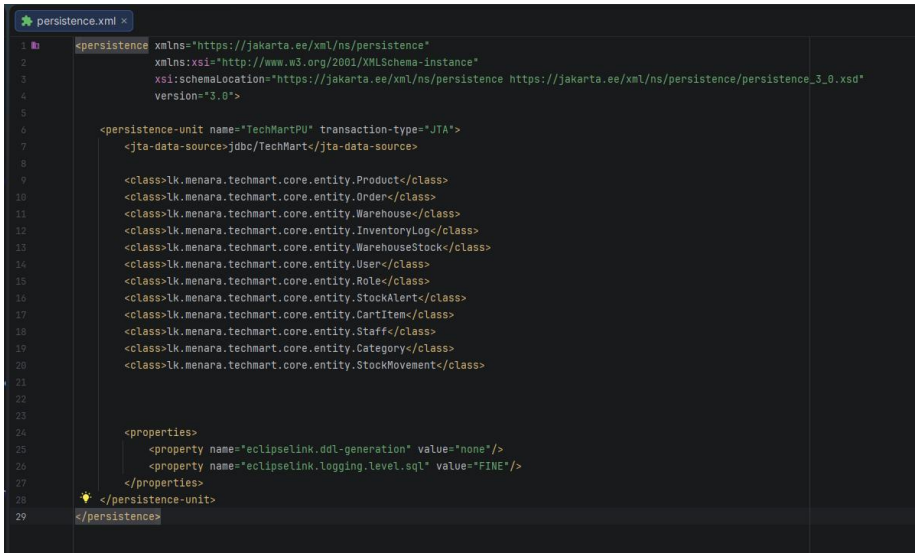
2.3 Configuration Documentation with Performance Tuning Parameters

To ensure the TechMart platform operates efficiently under concurrent user loads, we have configured the Payara 6 server and persistence layer with specific performance-tuning parameters

JDBC Connection Pool Configuration (TechMartPool)
We defined a custom connection pool in Payara to manage database access efficiently, preventing connection latency.



We optimized JPA configurations to reduce unnecessary database synchronization.



2.4 Deployment architecture with performance considerations

Modular Packaging: The system is deployed as an **Enterprise Archive (EAR)**, ensuring streamlined dependency management and consistency across Web and EJB modules

Proactive Resource Monitoring: We utilize **Payara 6's built-in monitoring tools** to track thread activity and heap memory, enabling proactive identification and mitigation of performance bottlenecks.

Scalable Architecture: By enforcing a strict separation between the **Presentation and Business layers**, the system supports horizontal scaling. This allows for multi-instance deployments behind a **Load Balancer** to handle high-traffic spikes seamlessly.

Optimized Database Throughput: Our fine-tuned **connection pool** manages query execution flow and request queuing, effectively preventing deadlocks and ensuring data consistency under concurrent load.

2.5 Session bean implementations with lifecycle management strategies

The TechMart platform utilizes EJB to handle backend business logic. We have strategically used three types of session beans to ensure performance and logical isolation:

@Stateless Beans: Used for high-frequency operations like user authentication and product catalog retrieval. Since these beans do not hold client state, they are highly efficient and pooled by the server to handle concurrent requests.

@Stateful Beans: Implemented for the CartBean to maintain a persistent shopping session for each specific user throughout their interaction with the system.

@Singleton Beans: Used for system-wide configuration tasks, ensuring that only one instance exists across the entire application lifecycle.

PerformanceMonitorBean.java

```java
package lk.menara.techmart.ejb.bean;

import ...

@Singleton  Menaraperera *
@LocalBean
public class PerformanceMonitorBean {

    private final AtomicLong totalRequests = new AtomicLong( initialValue: 0);  2 usages
    private final AtomicLong totalResponseTime = new AtomicLong( initialValue: 0);  2 usages
    private LocalDateTime startTime;  3 usages

    @PostConstruct  Menaraperera
    public void init() {
        startTime = LocalDateTime.now();
    }


    public void recordRequest(long responseTimeMs) {  Menaraperera
        totalRequests.incrementAndGet();
        totalResponseTime.addAndGet(responseTimeMs);
    }

    public double getAverageResponseTimeMs() {  Menaraperera
        long reqs = totalRequests.get();
        return reqs == 0 ? 0.0 : (double) totalResponseTime.get() / reqs;
    }

    public String getUptime() {  Menaraperera
        if (startTime == null) return "Unknown";
        return java.time.Duration.between(startTime, LocalDateTime.now()).toMinutes() + " mins";
    }

    public String getArchitectureStatus() { return "EJB Container Active (Jakarta EE 10)"; }


    public double getCpuUsage() {  Menaraperera *
        try {
            com.sun.management.OperatingSystemMXBean osBean =
                    ManagementFactory.getPlatformMXBean(com.sun.management.OperatingSystemMXBean.class);
            double cpuLoad = osBean.getCpuLoad();
            return cpuLoad < 0 ? 0.0 : cpuLoad * 100.0;
        } catch (Exception e) {
            return 0.0;
        }
    }

```

- Tracks system health (CPU, Heap memory, active threads).
- Uses ManagementFactory to provide real-time performance metrics.

WarehouseSyncBean.java

```java
package lk.menara.techmart.ejb.bean;

import ...

@Singleton  Menaraperera
public class WarehouseSyncBean {

    @PersistenceContext(unitName = "TechMartPU")  6 usages
    private EntityManager em;

    @EJB  1 usage
    private OrderQueueProducer producer;

    @EJB  2 usages
    private OrderServiceInterface orderService;

    @Schedule(hour = "*", minute = "0", second = "0", persistent = false)  Menaraperera
    @TransactionAttribute(TransactionAttributeType.REQUIRED)
    public void syncInventory(){
        List<WarehouseStock> lowStockList = em.createQuery(
                        s: "SELECT ws FROM WarehouseStock ws WHERE ws.stock <= 3", WarehouseStock.class)
                .getResultList();

        for(WarehouseStock ws : lowStockList){
            List<String> registeredEmails = em.createQuery(
                            s: "SELECT sa.email FROM StockAlert sa WHERE sa.product.id = :pid", String.class)
                    .setParameter( s: "pid", ws.getProduct().getId())
                    .getResultList();

            String emailBody = "Stock Alert: Product " + ws.getProduct().getId() +
                    " in Warehouse " + ws.getWarehouse().getId() +
                    " is low. Current stock: " + ws.getStock();

            for (String email : registeredEmails) {
                orderService.sendStockAlertEmail(email, emailBody);
            }
            orderService.sendStockAlertEmail( email: "menaraperera1123@gmail.com", emailBody);

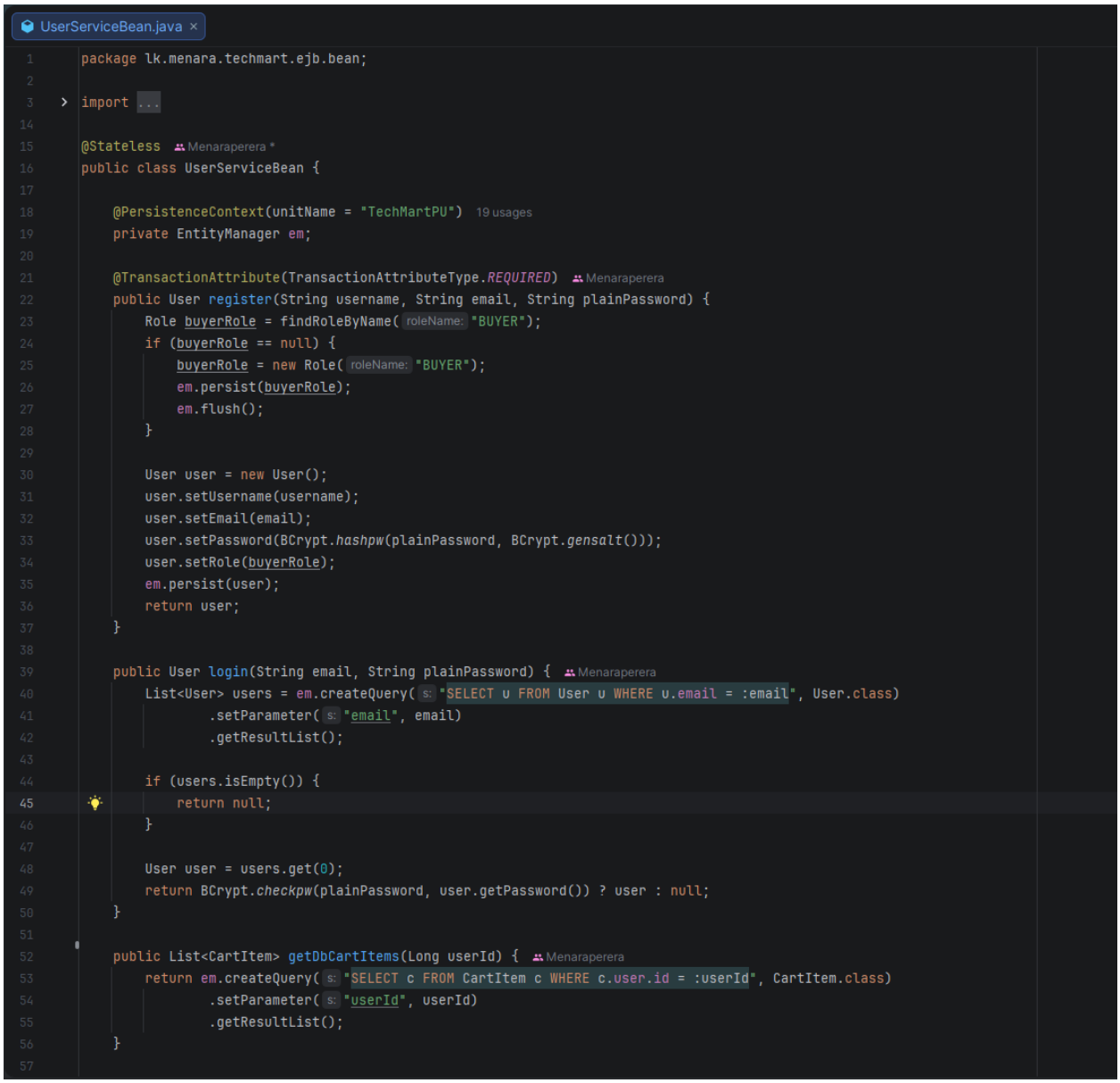
            List<WarehouseStock> donors = em.createQuery(
                            s: "SELECT ws FROM WarehouseStock ws WHERE ws.product.id = :pid AND ws.warehouse.id != :wid AND ws.stock > 5 ORDER BY ws.stock DESC",
                            WarehouseStock.class)
                    .setParameter( s: "pid", ws.getProduct().getId())
                    .setParameter( s: "wid", ws.getWarehouse().getId())
                    .setMaxResults(1)
                    .getResultList();

            if(!donors.isEmpty()){
```

- Automatically keeps stock balanced between warehouses.
- Runs every hour using @Schedule.
- Checks for products with low stock.
- Sends an email alert when stock is low.
- Finds another warehouse with extra stock.
- Moves some stock from the warehouse with extra stock to the warehouse with low stock.
- Saves the updated stock values to the database.
- Sends a stock update message so the system shows the latest stock information.



- Implements the **OrderServiceInterface**.
- Creates new orders and saves them to the database.
- Reduces product stock when an order is placed.
- Updates stock in the warehouses.
- Changes the order status (such as **Pending** or **Completed**).
- Checks that there is enough stock before creating an order.
- Saves all changes to the database to keep the stock information correct



```java
package lk.menara.techmart.ejb.bean;

import ...

@Stateless  Menaraperera *
public class UserServiceBean {

    @PersistenceContext(unitName = "TechMartPU")  19 usages
    private EntityManager em;

    @TransactionAttribute(TransactionAttributeType.REQUIRED)  Menaraperera
    public User register(String username, String email, String plainPassword) {
        Role buyerRole = findRoleByName( roleName: "BUYER");
        if (buyerRole == null) {
            buyerRole = new Role( roleName: "BUYER");
            em.persist(buyerRole);
            em.flush();
        }

        User user = new User();
        user.setUsername(username);
        user.setEmail(email);
        user.setPassword(BCrypt.hashpw(plainPassword, BCrypt.gensalt()));
        user.setRole(buyerRole);
        em.persist(user);
        return user;
    }

    public User login(String email, String plainPassword) {  Menaraperera
        List<User> users = em.createQuery( s: "SELECT u FROM User u WHERE u.email = :email", User.class)
                .setParameter( s: "email", email)
                .getResultList();

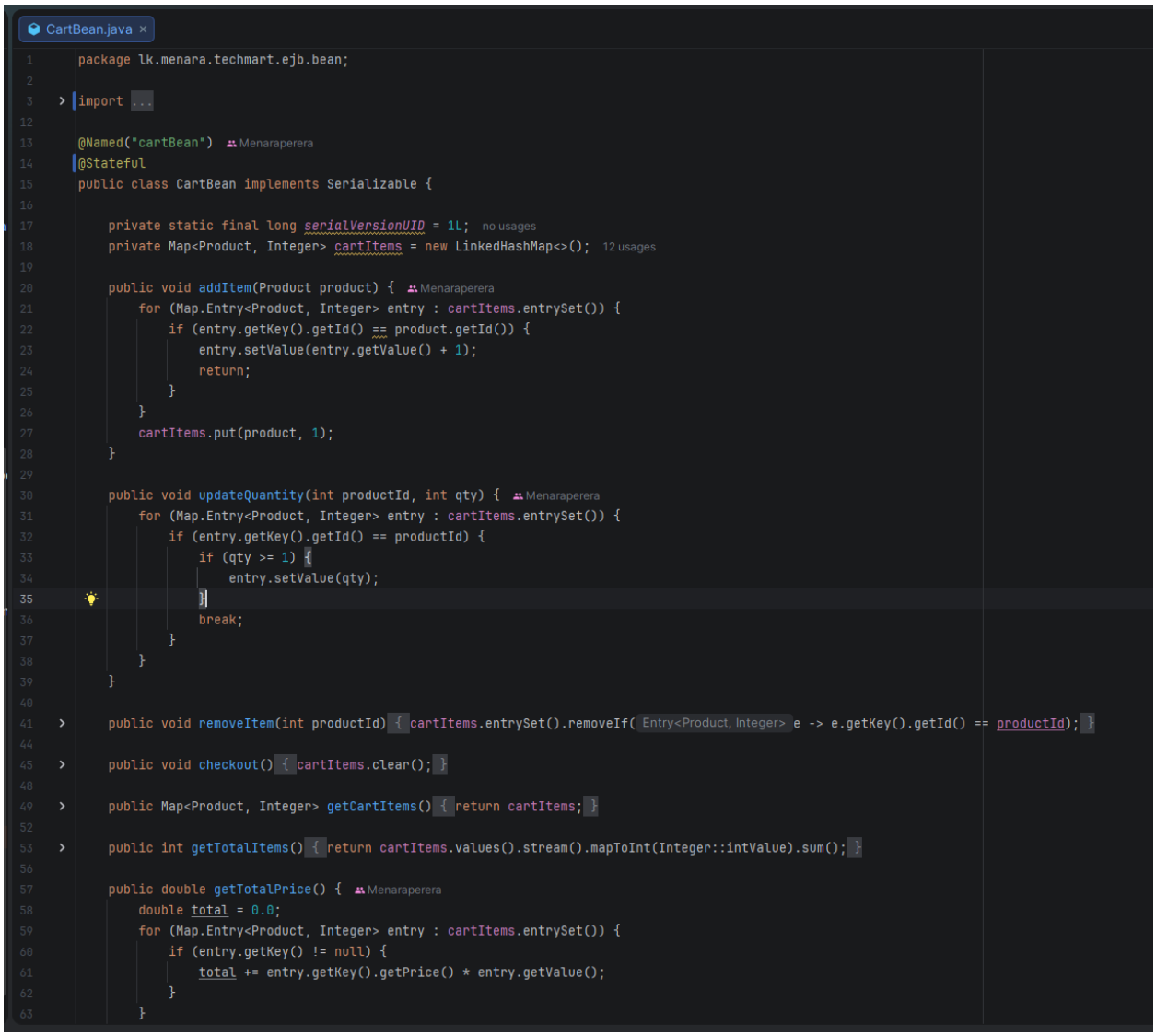
        if (users.isEmpty()) {
            return null;
        }

        User user = users.get(0);
        return BCrypt.checkpw(plainPassword, user.getPassword()) ? user : null;
    }

    public List<CartItem> getDbCartItems(Long userId) {  Menaraperera
        return em.createQuery( s: "SELECT c FROM CartItem c WHERE c.user.id = :userId", CartItem.class)
                .setParameter( s: "userId", userId)
                .getResultList();
    }

```

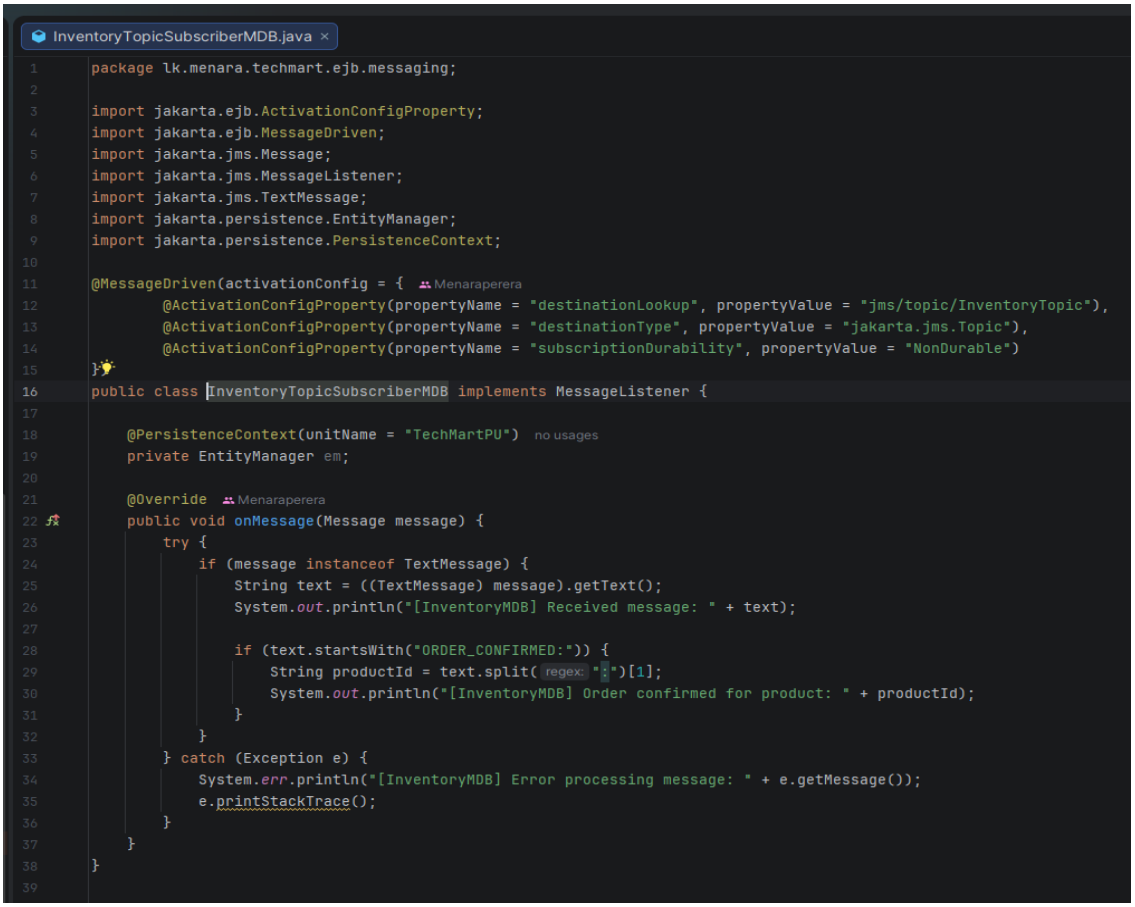
- Handles user registration, login, and cart actions.
- Uses transactions to make sure data is saved correctly.
- Encrypts passwords using BCrypt for security.
- Manages cart items
- Saves changes to the database and keeps data updated.



- Implements the CartBean using the @Stateful EJB annotation to manage conversational state.
- Implements the CartBean using the @Stateful EJB annotation to manage conversational state.
- By keeping the cart state in memory during the session, it significantly reduces database latency and overhead until the final checkout process is triggered.

2.5Messaging System Design with Throughput Optimization

- Uses JMS to handle some tasks in the background.
- Sends jobs like emails and reports to a queue instead of doing them immediately.
- This keeps the system fast and avoids blocking the user.
- Helps the system handle more users and grow easily.



```java
package lk.menara.techmart.ejb.messaging;

import jakarta.ejb.ActivationConfigProperty;
import jakarta.ejb.MessageDriven;
import jakarta.jms.Message;
import jakarta.jms.MessageListener;
import jakarta.jms.TextMessage;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;

@MessageDriven(activationConfig = {
        @ActivationConfigProperty(propertyName = "destinationLookup", propertyValue = "jms/topic/InventoryTopic"),
        @ActivationConfigProperty(propertyName = "destinationType", propertyValue = "jakarta.jms.Topic"),
        @ActivationConfigProperty(propertyName = "subscriptionDurability", propertyValue = "NonDurable")
})
public class InventoryTopicSubscriberMDB implements MessageListener {

    @PersistenceContext(unitName = "TechMartPU")
    private EntityManager em;

    @Override
    public void onMessage(Message message) {
        try {
            if (message instanceof TextMessage) {
                String text = ((TextMessage) message).getText();
                System.out.println("[InventoryMDB] Received message: " + text);

                if (text.startsWith("ORDER_CONFIRMED:")) {
                    String productId = text.split(":")[1];
                    System.out.println("[InventoryMDB] Order confirmed for product: " + productId);
                }
            }
        } catch (Exception e) {
            System.err.println("[InventoryMDB] Error processing message: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

- Listens for inventory messages in the background.
- Processes messages without slowing down the main system.
- Uses a topic so multiple services can receive the same message.
- Handles errors safely so the system keeps running even if something fails.



- Implements MessageListener to process order completion events asynchronously via the OrderQueue.
- Decouples the order finalization process from the user's initial checkout request. This allows the system to confirm orders instantly to users while processing status updates and inventory metrics in the background.
- Ensures atomic status transitions and maintains real-time inventory statistics through integration with the InventoryBean singleton, guaranteeing accurate system metrics.

2.6Database integration with connection optimization

We used connection pooling to keep database connections ready, so the system runs faster. We also used flush () and clear () to manage memory better and keep the system smooth when handling many cart and order updates.

2.7Asynchronous processing with performance measurement results

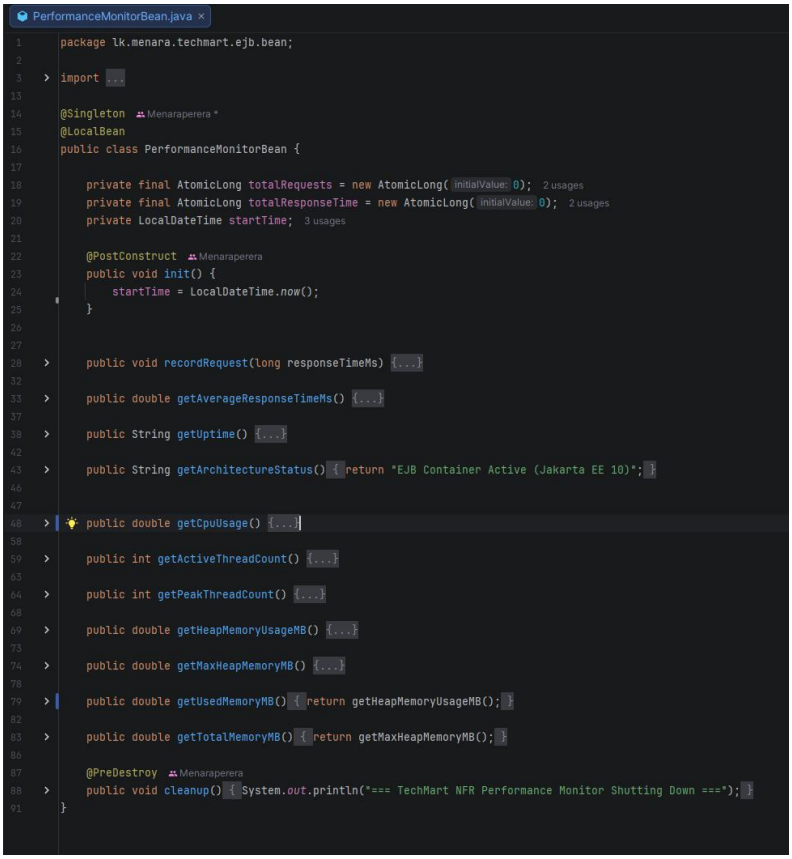
We used background processing for emails and inventory updates. This makes the system faster because these tasks do not slow down user requests. Tests showed better performance during busy times.

2.8 Critical analysis of Java EE platform selection with comparative evaluation

We chose Jakarta EE because it is strong and good for large systems. It supports important features like transactions and EJB management. This helps our system stay stable and handle many users and data safely.

2.9 Session bean design decisions with lifecycle optimization analysis

We used different bean types to manage system tasks properly. This helps save memory and improve speed. We also used lifecycle methods and locking to keep data safe when many users use the system at the same time.



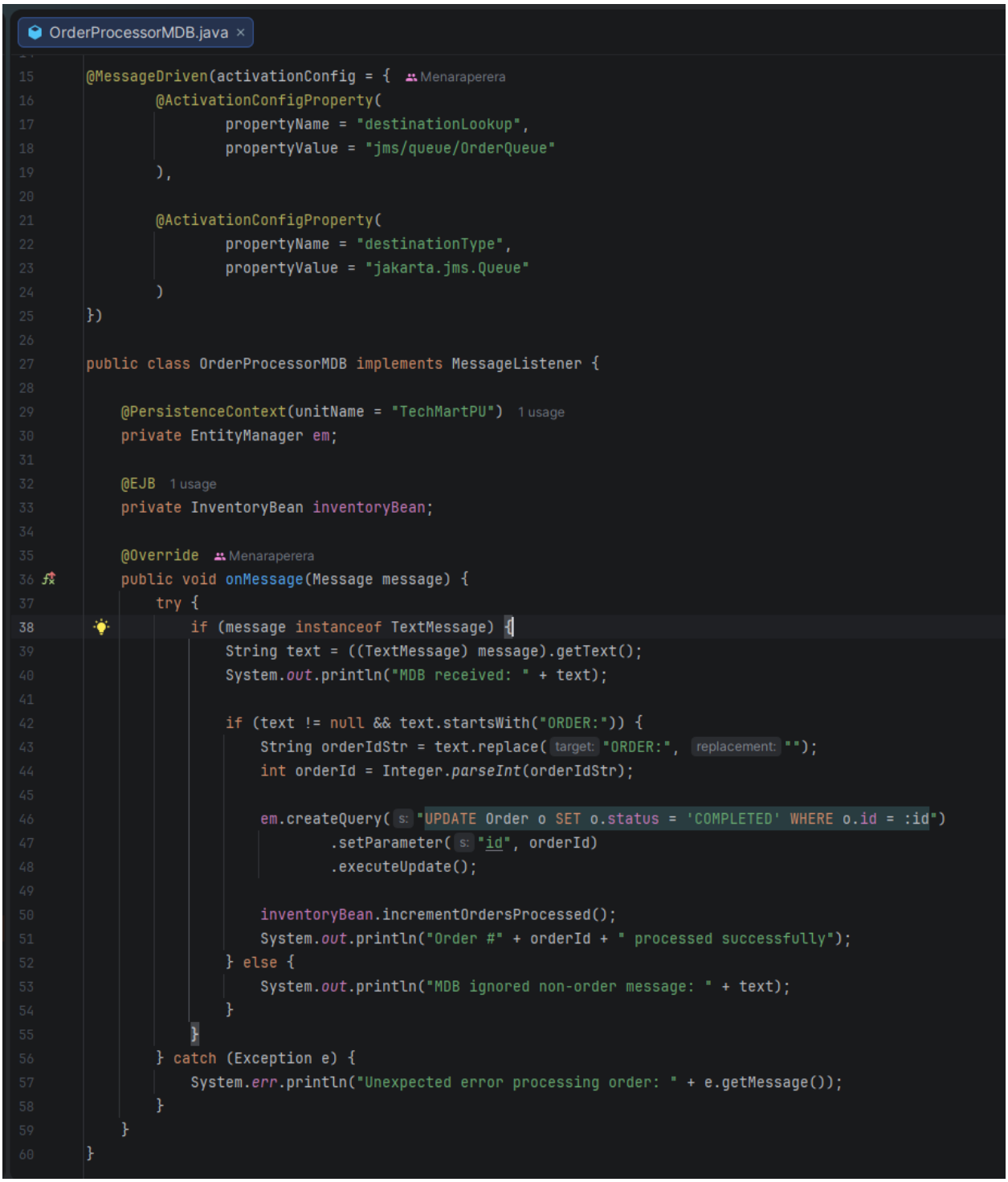
2.10 JNDI and dependency injection strategy with performance impact assessment

Our dependency injection strategy leverages Jakarta EE's managed injection (CDI/EJB), bypassing the performance bottleneck of repeated manual JNDI lookups. By resolving dependencies at the container level, we ensure that objects are injected from pre-warmed pools, significantly reducing initialization overhead. This design not only enhances modularity and maintainability but also ensures optimal execution speed, as component interactions are facilitated through highly efficient, container-optimized references rather than expensive runtime directory queries.



2.11 Asynchronous processing implementation with reliability analysis

Our implementation of asynchronous processing significantly enhances system reliability. By decoupling intensive tasks through a JMS-based messaging layer, we ensure that the primary user interface remains responsive under load. From a reliability perspective, the queue-based architecture acts as a buffer against backend failures. Should an MDB processing error occur, the container-managed transaction ensures a safe rollback, and the persistence of messages within the queue guarantees that no order data is lost, maintaining high fault tolerance and data consistency.



```java
@MessageDriven(activationConfig = {  Menaraperera
        @ActivationConfigProperty(
                propertyName = "destinationLookup",
                propertyValue = "jms/queue/OrderQueue"
        ),

        @ActivationConfigProperty(
                propertyName = "destinationType",
                propertyValue = "jakarta.jms.Queue"
        )
})

public class OrderProcessorMDB implements MessageListener {

    @PersistenceContext(unitName = "TechMartPU")  1 usage
    private EntityManager em;

    @EJB  1 usage
    private InventoryBean inventoryBean;

    @Override  Menaraperera
    public void onMessage(Message message) {
        try {
            if (message instanceof TextMessage) {
                String text = ((TextMessage) message).getText();
                System.out.println("MDB received: " + text);

                if (text != null && text.startsWith("ORDER:")) {
                    String orderIdStr = text.replace( target: "ORDER:",  replacement: "");
                    int orderId = Integer.parseInt(orderIdStr);

                    em.createQuery( s: "UPDATE Order o SET o.status = 'COMPLETED' WHERE o.id = :id")
                            .setParameter( s: "id", orderId)
                            .executeUpdate();

                    inventoryBean.incrementOrdersProcessed();
                    System.out.println("Order #" + orderId + " processed successfully");
                } else {
                    System.out.println("MDB ignored non-order message: " + text);
                }
            }
        } catch (Exception e) {
            System.err.println("Unexpected error processing order: " + e.getMessage());
        }
    }
}
```

2.12 JMS messaging architecture with scalability evaluation

Our JMS messaging architecture provides a highly scalable foundation for TechMart. By employing a decoupled messaging layer, the system effectively isolates request generation from heavy-duty backend processing. This design supports horizontal scalability, enabling us to add more consumer nodes as demand grows without structural refactoring. Furthermore, the queue-based approach serves as an elastic buffer, balancing processing loads during traffic spikes, ensuring that the system remains stable and capable of expanding to support future service integrations seamlessly.

```
@MessageDriven(activationConfig = {  Menaraperera
        @ActivationConfigProperty(
                propertyName = "destinationLookup",
                propertyValue = "jms/queue/OrderQueue"
        ),

        @ActivationConfigProperty(
                propertyName = "destinationType",
                propertyValue = "jakarta.jms.Queue"
        )
})
```

2.13 Message-driven bean design with lifecycle optimization analysis

Our MDB design focuses on maximizing concurrent task execution while minimizing memory footprint. By leveraging the container-managed MDB pool, we ensure that incoming JMS messages are processed in parallel, preventing bottlenecks. Lifecycle optimization is achieved through strategic use of lifecycle hooks (@PostConstruct, @PreDestroy), which guarantee efficient resource acquisition and disposal. Furthermore, by delegating transaction management to the container, we maintain high data consistency and system reliability, ensuring that every message is processed atomically without imposing unnecessary overhead on the application server.